

SESSION 1: BECOMING A FULL-STACK CREATOR

Capstone Project — Full-Stack Web Application Student Guide & Architecture Worksheet

Course: Full-Stack Web Development
Capstone

Module: 01 - Architecture &
Blueprinting

Target Audience: Junior Tech Creators /
App CEOs

1. Introduction: Be the CEO of Your Own App Startup

Welcome to your journey as an app creator! In this Capstone Project, you are taking on the role of the **CEO of your own App Startup**. Imagine creating a modern application that solves real-world problems in your local community, such as connecting neighbors to share delicious home-cooked meals.

To transform a creative idea into a fully functional product, you need to think like a system architect and understand how all pieces of technology work together behind the scenes.



Startup Challenge Idea

NeighbourBites: An interactive platform where home chefs can list today's special home-cooked dishes, and local neighbors can discover, request, and receive fresh meals. How would you design the user screens, store the recipes, and track deliveries in real time?

2. Understanding Full-Stack Development: The Whole Package

Full-Stack development combines two primary pillars: the visible **Frontend** that users interact with directly, and the hidden **Backend** that processes data and executes application logic.

FRONTEND The Visible Interface

Everything the user sees, clicks, and touches on their screen. Built using HTML, CSS, and interactive visual designs.

- User Login Screens
- Interactive Meal Lists & Buttons
- Visual Delivery Animations

BACKEND The Hidden Engine

The server-side logic and database operations operating under the hood. It validates users, processes requests, and handles secure data.

- Authentication & Passwords
- Order Status Calculations
- Database Queries & Management

3. The Digital Blueprint: Key Tech Stack Components

Building an enterprise-grade web application requires multiple specialized technologies working in harmony. To master these concepts, let's examine their real-world analogies:

Technology	Real-World Analogy	Role in Application Architecture
HTML / CSS	Building Blocks & Paints	

Technology	Real-World Analogy	Role in Application Architecture
		Defines the visual structure, layout, typography, colors, and interactive user interface of the Frontend.
API (Application Programming Interface)	Roadside Restaurant Waiter	Carries user order requests from the Frontend table down to the Backend kitchen, and returns the response back to the user.
Database	Digital 'Godown' / Warehouse	Stores structured information—such as user profiles, recipes, and order history—safely for retrieval.
WebSockets	Live Walkie-Talkie Channel	Provides continuous two-way communication to deliver real-time updates like live driver location tracking.
Cloud Storage	Online Security Vault	Saves high-resolution media, dish photos, and document files safely online for fast global access.

4. Recap: The MVC Architecture Pattern

Software engineers structure app code using the **Model-View-Controller (MVC)** pattern to keep code clean, scalable, and easy to maintain.

 **The Three Pillars of MVC**

- **Model (Data Layer):** Manages the database logic, structures application data, and handles saving/retrieving information.
- **View (Presentation Layer):** The visual user interface that renders data into readable displays and interactive forms for users.
- **Controller (Logic Layer):** The central brain that receives user inputs from the View, requests action from the Model, and determines what response to show back on the View.

5. Indian Tech Success Stories: Architecture in Action

Many leading technology startups rely on these exact full-stack building blocks to serve millions of users daily.

Startup	Core Category	Architectural Innovation & Functionality
Zomato	Food Delivery & Discovery	Revolutionized food ordering with menu browsing, dynamic pricing APIs, and live GPS order tracking via WebSockets.
Swiggy	Hyperlocal Delivery	Offers lightning-fast food and grocery delivery from local outlets to doorsteps with optimized routing algorithms.
Paytm	Digital Payments & Fintech	Simplifies digital wallet payments, QR code transactions, and secure banking integrations for millions.

6. Community App Ideas for Your Capstone Project

Here are 5 real-world community project ideas you can build using full-stack development:

01. Society Helper

Tracks gate entries, visitor logs, and water delivery schedules for gated residential societies.

02. Study Buddy

Real-time chat platform for student groups to request and offer live homework assistance.

03. Lassi Stall Tracker

Interactive map platform locating nearby authentic lassi stalls, user ratings, and daily special offers.

04. Eco-Warrior

Community eco-logger for recording plastic recycling efforts and awarding green badges.

05. RWA Notice Board

Digital bulletin board for Resident Welfare Associations to broadcast urgent announcements, event updates, and maintenance schedules.

7. Architecture Worksheet: Draw Your App Blueprint

Now it's time to act as CEO! Fill out this design blueprint worksheet to map out your Capstone App architecture.

App Concept Profile

App Title / Project Name: _____

Primary Goal & Problem Solved:

Tech Stack & System Architecture Plan

1. Frontend View (What will users see on screen?)

2. Database Model (What data tables and info will be saved in the 'Godown'?)

3. API & WebSockets Plan (Will your app need real-time live updates?)

4. Cloud Storage Plan (What media images or documents will be saved in the cloud?)

SECTION 8: STUDENT PRACTICE QUESTIONS

Session 1 Knowledge Evaluation — Answer all questions in the space provided.

Instructions: Test your understanding of Full-Stack Web Development, system architecture, and tech components. Write your responses clearly in the boxes below. (No answers included).

Q1. Differentiating Stack Pillars

Explain the difference between the **Frontend** and the **Backend** of a web application in your own words. Give one specific example of a component for each.

Q2. Analogy Application

Why is an **API** often compared to a *'waiter at a roadside restaurant'*? What role does it perform between the Frontend and the Backend?

Q3. Real-Time Technology Selection

If you were building a live food delivery application like Swiggy or Zomato, why would you choose **WebSockets** over regular standard database queries for tracking driver delivery positions?

Q4. MVC Pattern Components

Identify which component of the **Model-View-Controller (MVC)** pattern is responsible for each of the following tasks:

- (a) Displaying a list of nearby Lassi stalls on the smartphone screen: _____
- (b) Saving a user's recycling score into the Eco-Warrior database: _____
- (c) Checking if a user has entered the correct login password: _____

Q5. Startup System Architecture Design

Suppose you are building the **Society Helper** app. List two types of data that your app's **Database ('Godown')** must store, and describe one feature on the **Frontend View** that society residents will interact with.

Q6. Cloud Storage vs Database Storage

Explain why user photos or dish images are typically stored in **Cloud Storage** rather than directly inside a relational text database.